

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

In the name of Allah, the most gracious, the most merciful



BY:

HAFIZ MUHAMMAD ASAD MUSTAFA

Phone: +92-303-2554632

To-mail: hafizasad2554@gmail.com

TABLE OF CONTANT

Introduction

- Overview of the Project
- Objectives

Setup and Installation

- Requirements
- Installation Steps

Basic Concepts

- Data Types
- Variables
- Operators

Control Structures

- Conditional Statements (if, else, switch)
- Loops (for, while, do-while)

Functions

- Function Declaration and Definition
- Parameters and Return Types
- Function Overloading

Arrays and Pointers

- Array Declaration and Initialization
- Pointer Basics
- Pointer Arithmetic

Object-Oriented Programming (OOP)

- Classes and Objects
- Inheritance
- Polymorphism
- Encapsulation

Standard Template Library (STL)

- Containers (Vectors, Lists, Maps)
- Algorithms (Sorting, Searching)
- Iterators

File Handling

- Input/Output Operations
- File Streams

Exception Handling

- Try-Catch Blocks
- Throwing Exceptions

Memory Management

- Dynamic Memory Allocation (new/delete, malloc/free)
- Smart Pointers

Advanced Topics

- Templates
- Lambda Expressions
- Multithreading

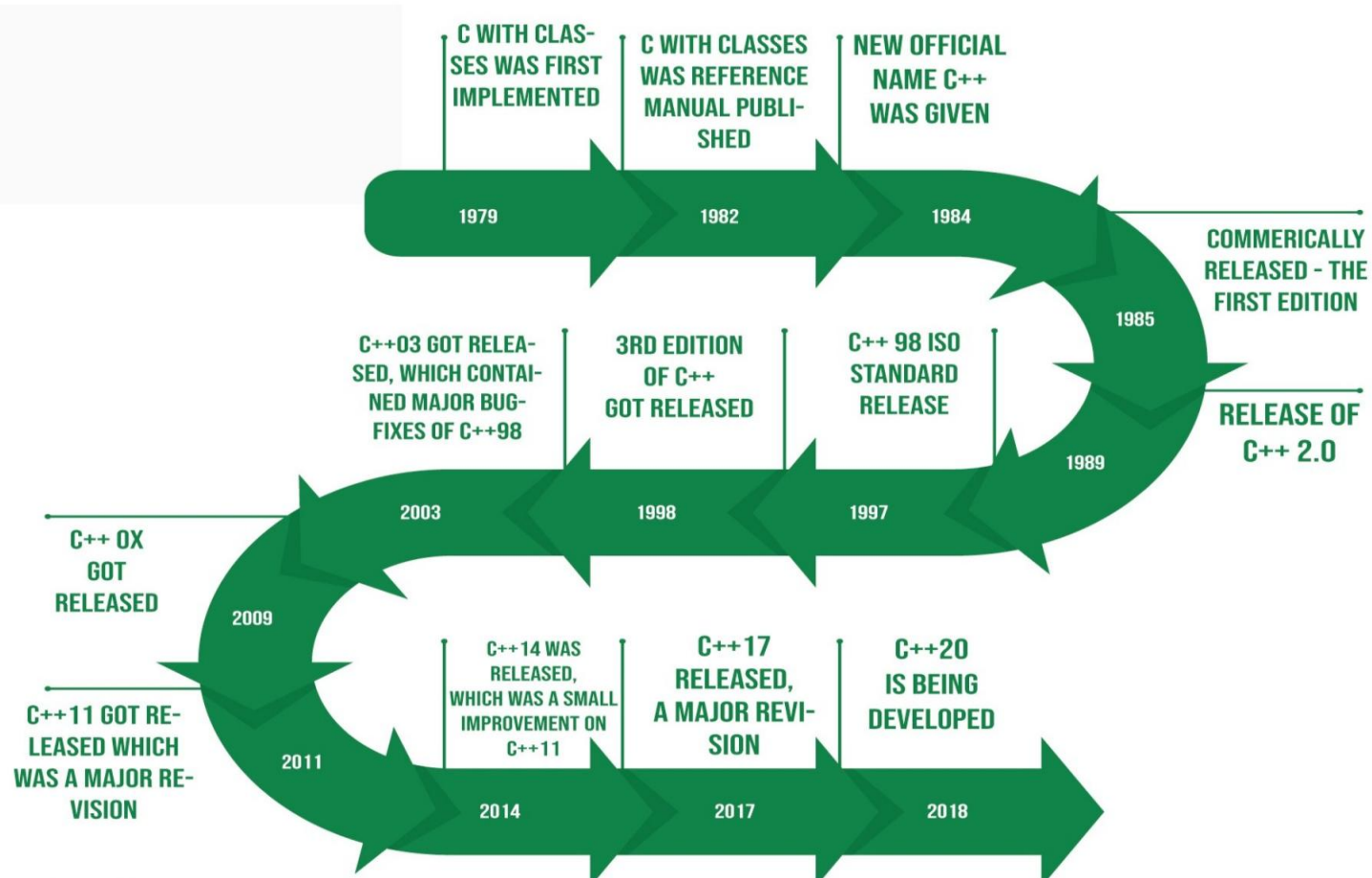
Best Practices

- Coding Standards
- Debugging Techniques
- Optimization Tips

Conclusion

- Summary
- Future Scope

History of C++

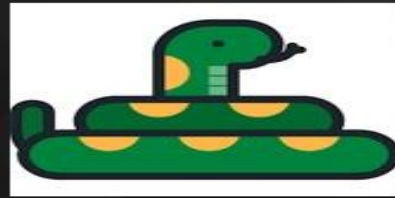


C++ PROJECTS IDEAS

EASY



**TIC TAC
TOE**



**SNAKE
GAME**



**QUIZ
GAME**



**FLAPPY
BIRD**

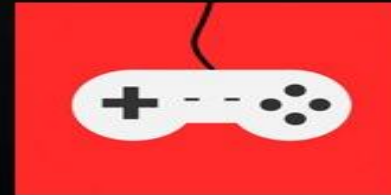
MID



**SHOOTER
GAME**



**CHESS
GAME**



**KBC
GAME**



**ANDROID
APP**

PRO



**WINDOWS
APP**



CHECKER

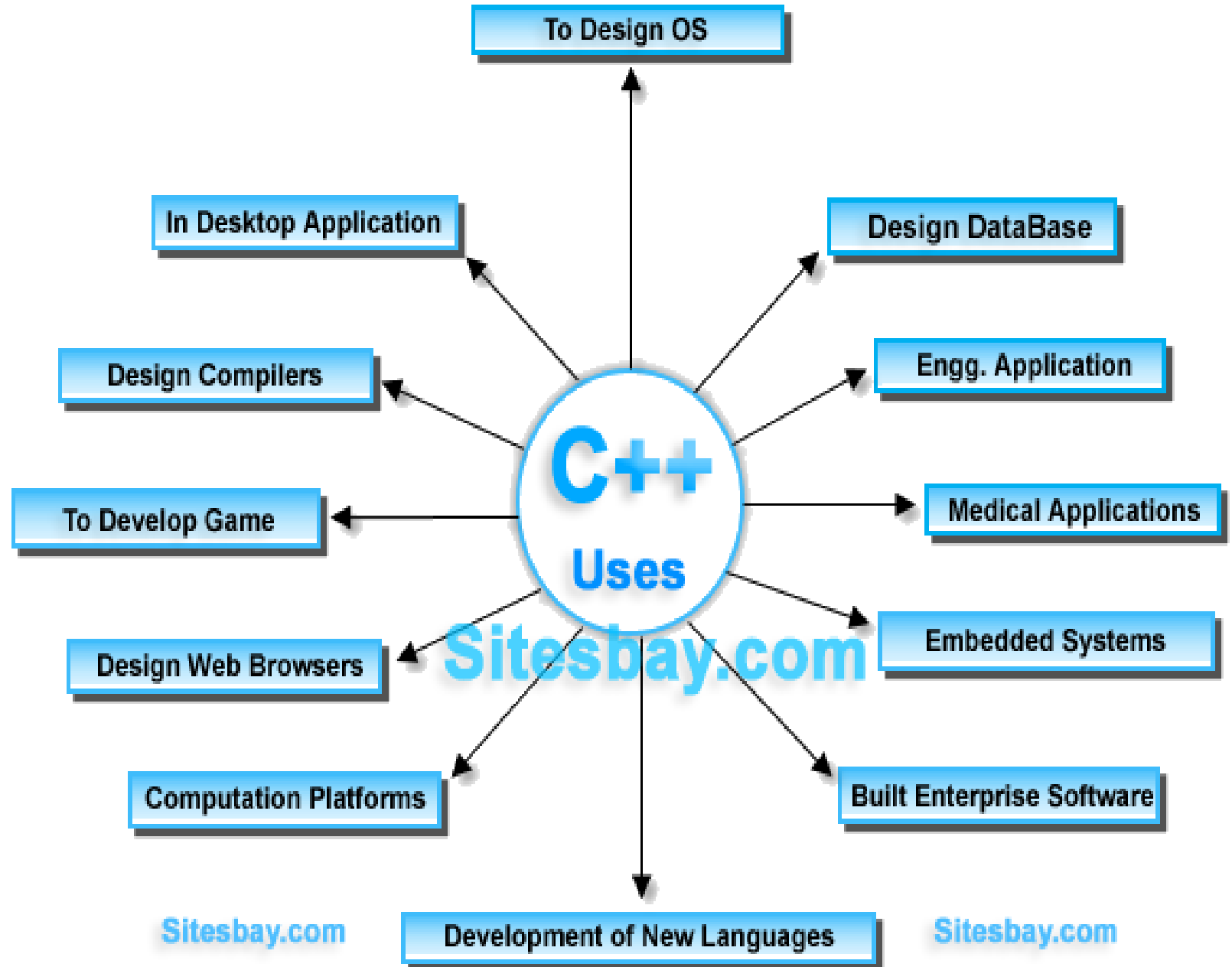


**CONTROLL-
ING
DEVICE**

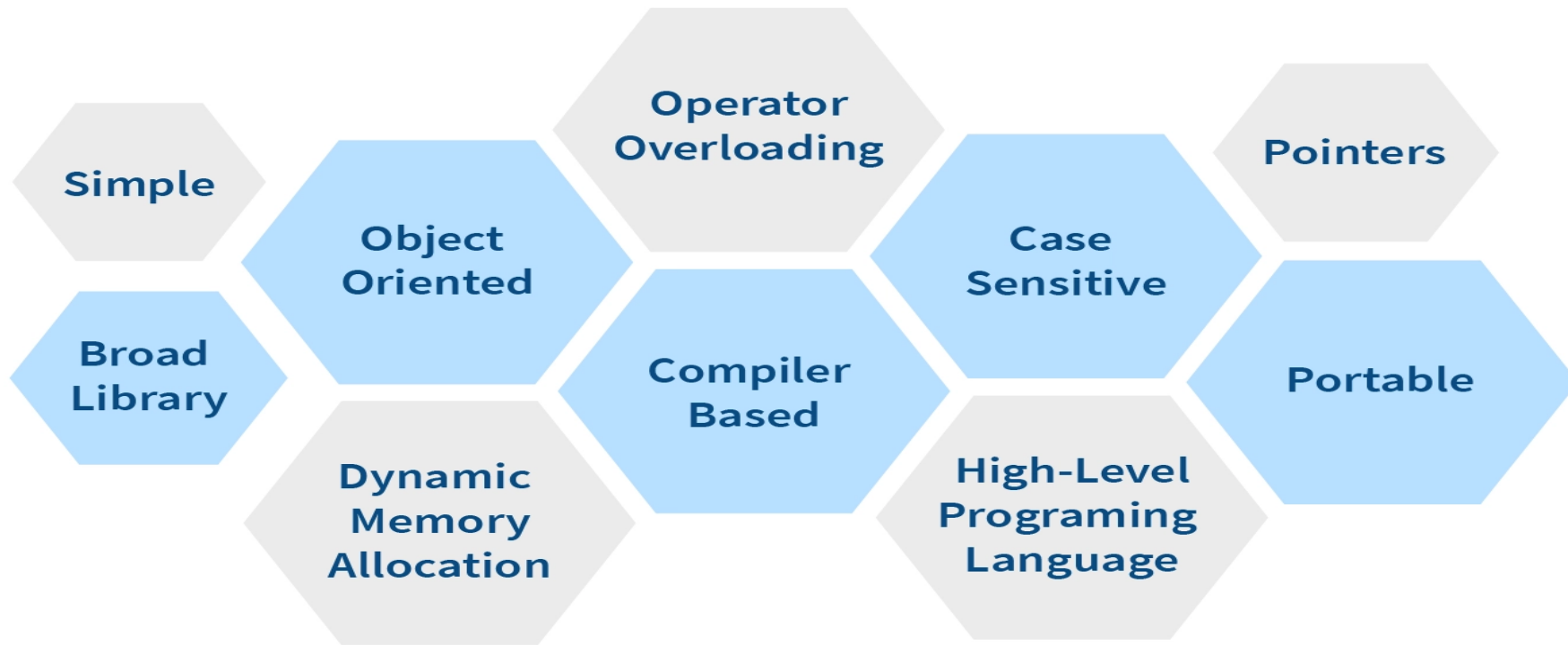


**UNI
SYST**

Why are
we
studying
C++



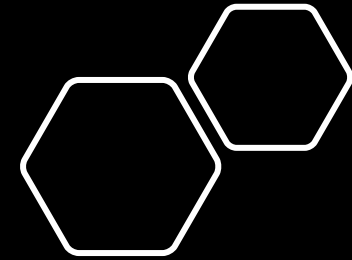
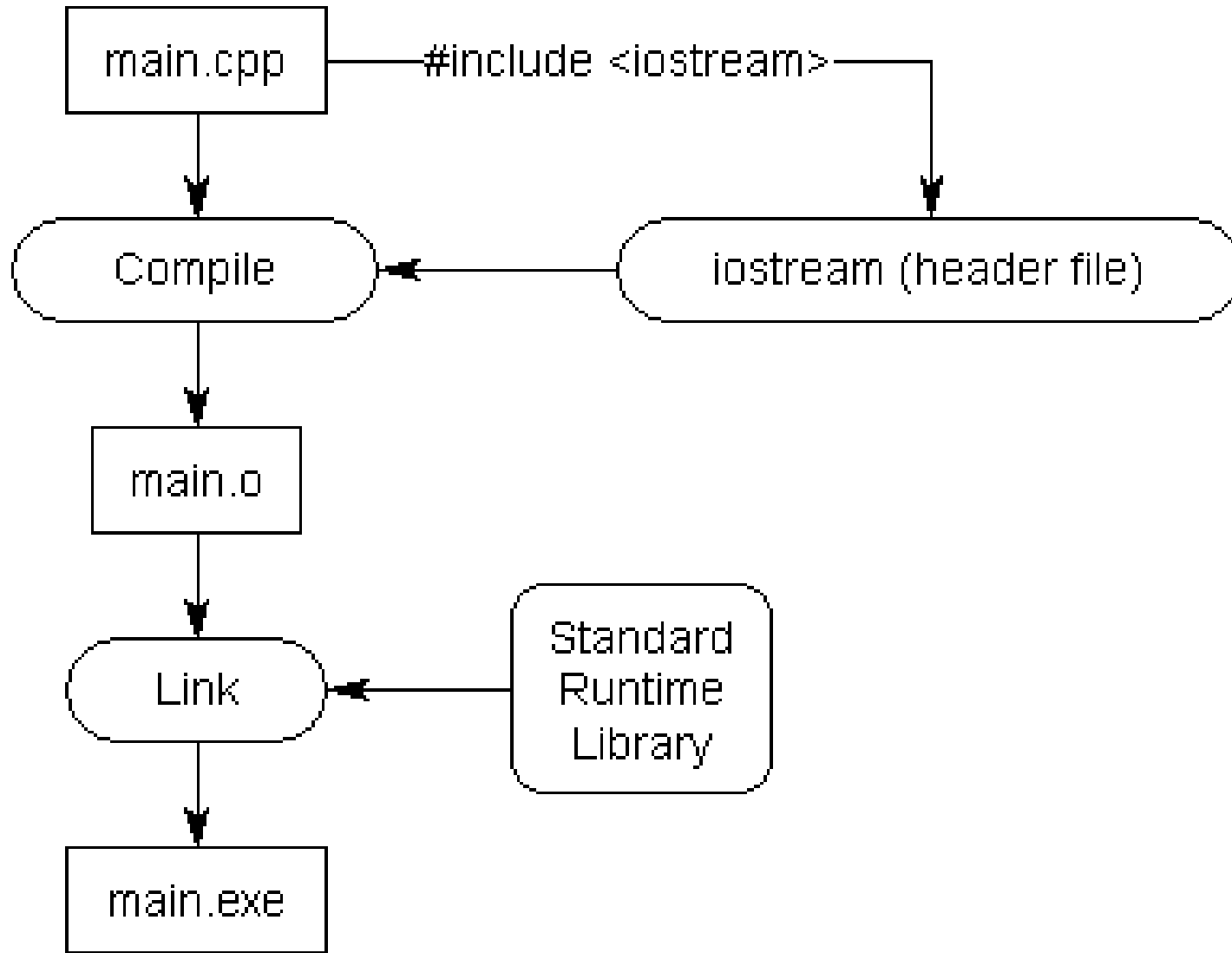
Features of C++



The Parts of a C++ Program

```
// sample C++ program ← comment
#include <iostream> ← preprocessor directive
using namespace std; ← which namespace to use
int main() ← beginning of function named main
{ ← beginning of block for main
    cout << "Hello, there!"; ← output statement
    return 0; ← send 0 to operating system
} ← end of block for main
```

← string literal



C Header files

A separate file that contains a related set of:

- Function prototypes
- typedef declarations
- #define constants
- etc.

File naming convention:

- All lowercase
- Use "_" to combine words
- ".h" is the file extension

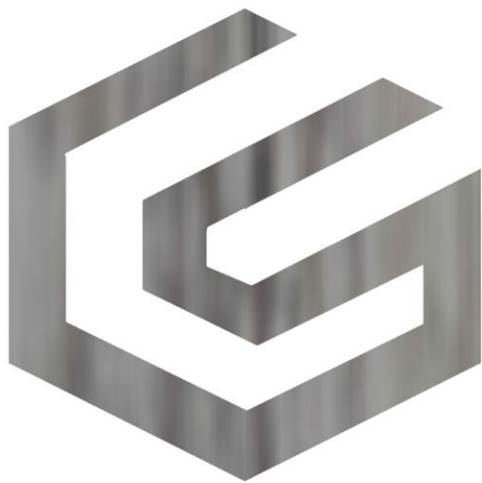
Example: atd_helper.h

You must "wrap" the header in a #ifndef to prevent circular inclusions (also called guarding)

```
#ifndef _ATD_HELPER_H_
#define _ATD_HELPER_H_

// Your header file code (typedefs, function prototypes, #defines, etc.)
// Use good comment headers to define each function (see example)
// ...

#endif // _ATD_HELPER_H
```



360

Standard C Library



stdio.h



math.h



conio.h



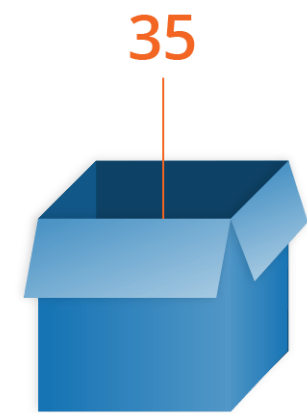
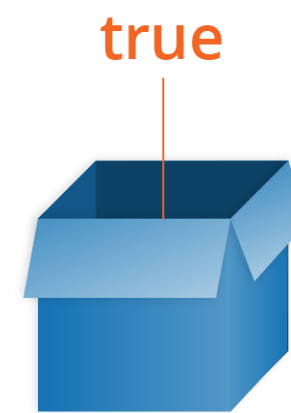
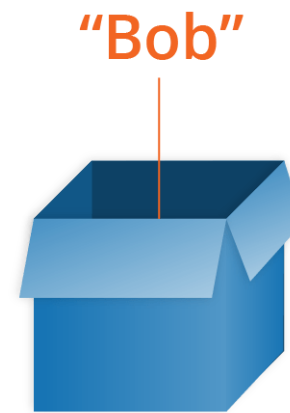
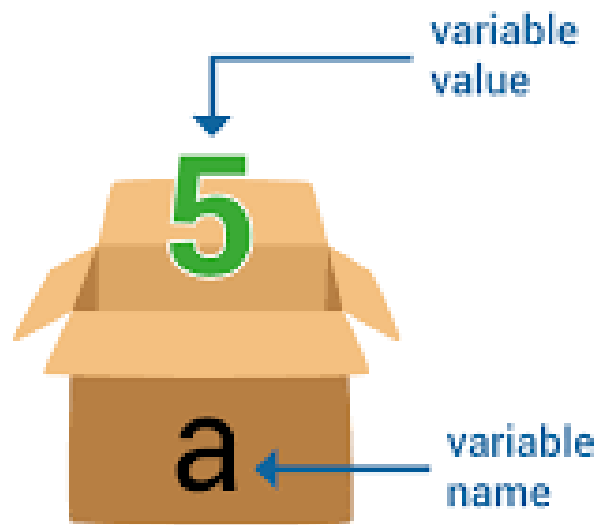
dos.h

not a complete list

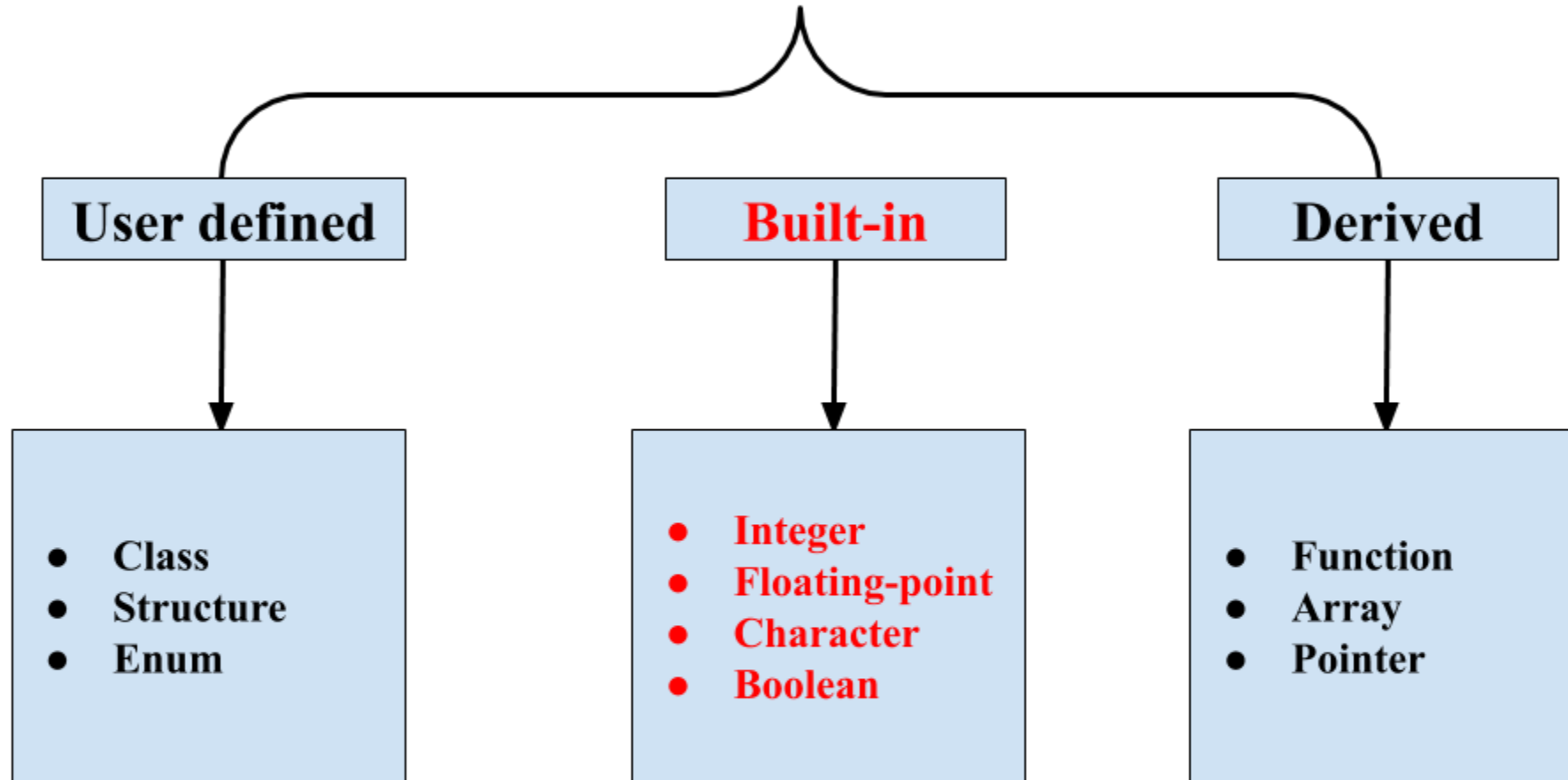
Sitesbay.com

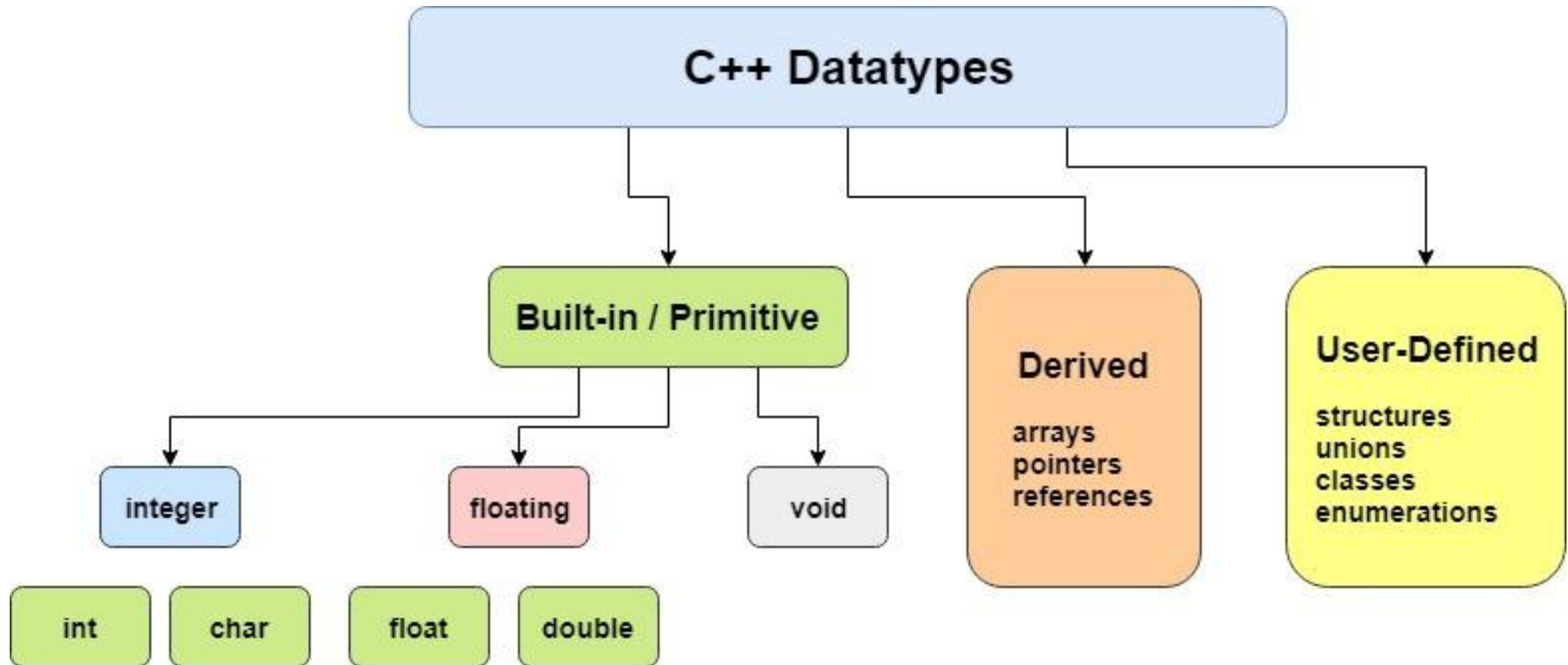
test.c Sitesbay.com

```
#include <stdio.h>  
printf("...");
```



Data Types in C++





Variable Declaration & initialization in one line

Data Type

Variable Name

Constant Value

int

age

=

20 ;



360



Container
named **age**
Holding a value
20

Arithmetic Operators

Operators	Meaning	Example	Result
+	Addition	4+2	6
-	Subtraction	4-2	2
*	Multiplication	4*2	8
/	Division	4/2	2
%	Modulus operator to get remainder in integer division	5%2	1
++	Increment	A = 10; A++	11
--	Decrement	A = 10; A--	9

Relational Operators

Operators	Meaning	Example	Result
<	Less than	$5 < 2$	False
>	Greater than	$5 > 2$	True
<=	Less than or equal to	$5 <= 2$	False
>=	Greater than or equal to	$5 >= 2$	True
==	Equal to	$5 == 2$	False
!=	Not equal to	$5 != 2$	True

Logical Operators

Operator	Meaning	Example	Result
&&	Logical and	$(5 < 2) \&\& (5 > 3)$	False
	Logical or	$(5 < 2) (5 > 3)$	True
!	Logical not	$!(5 < 2)$	True

||

Operand 1	Operand 2	Result
True	True	True
True	False	True
False	True	True
False	False	False

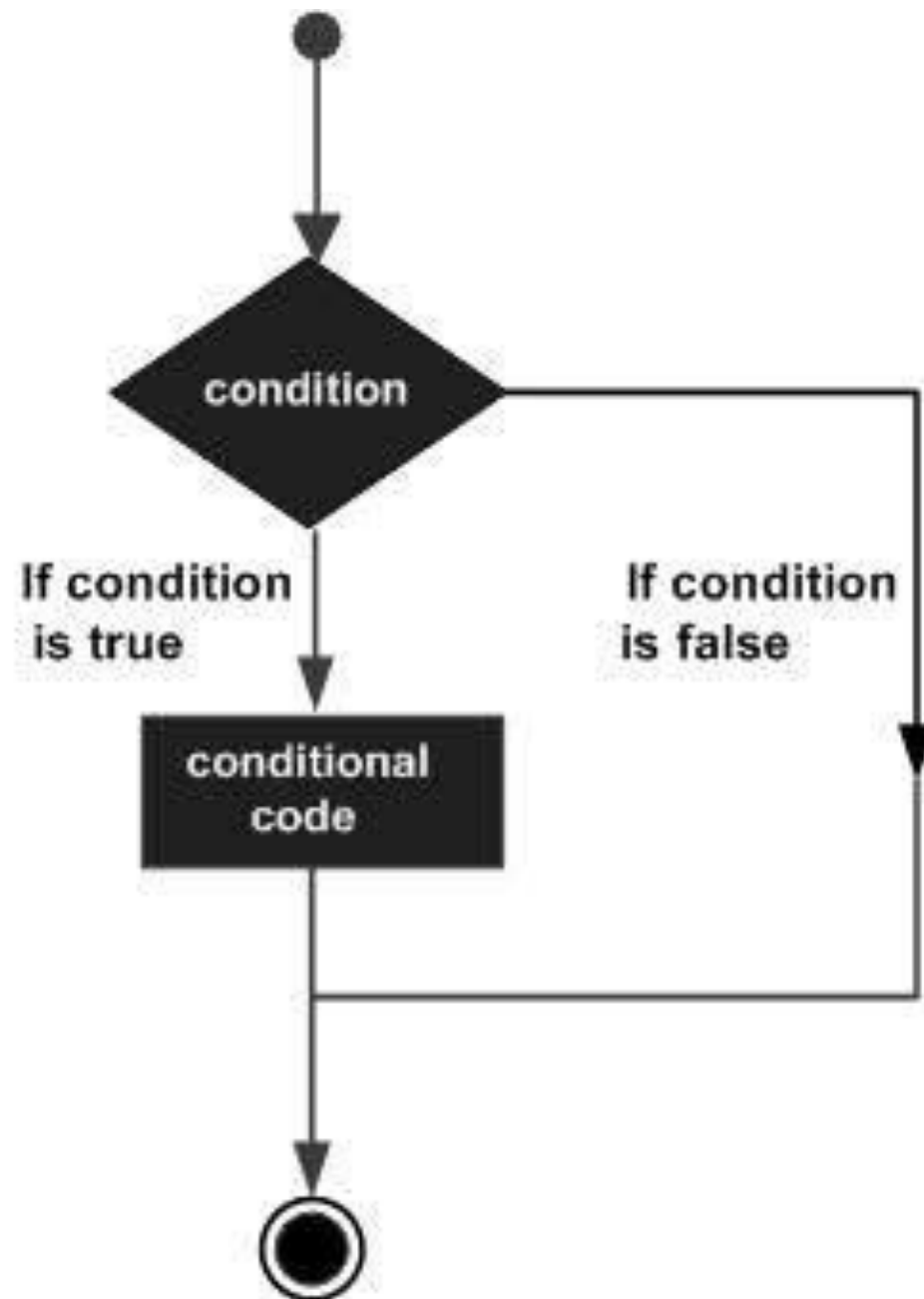
&&

Operand 1	Operand 2	Result
True	True	True
True	False	False
False	True	False
False	False	False

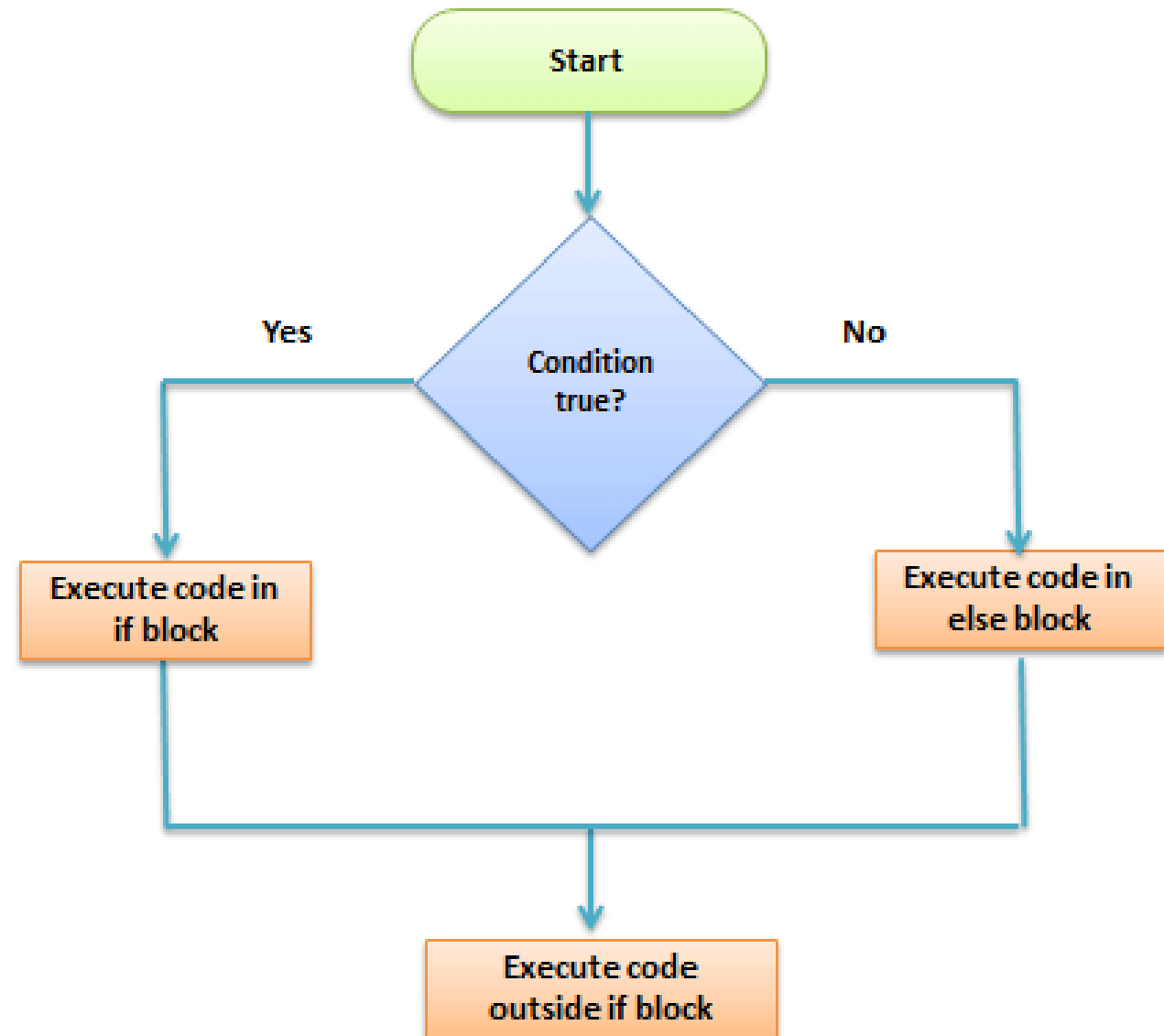
!

Operand	Result
False	True
True	False

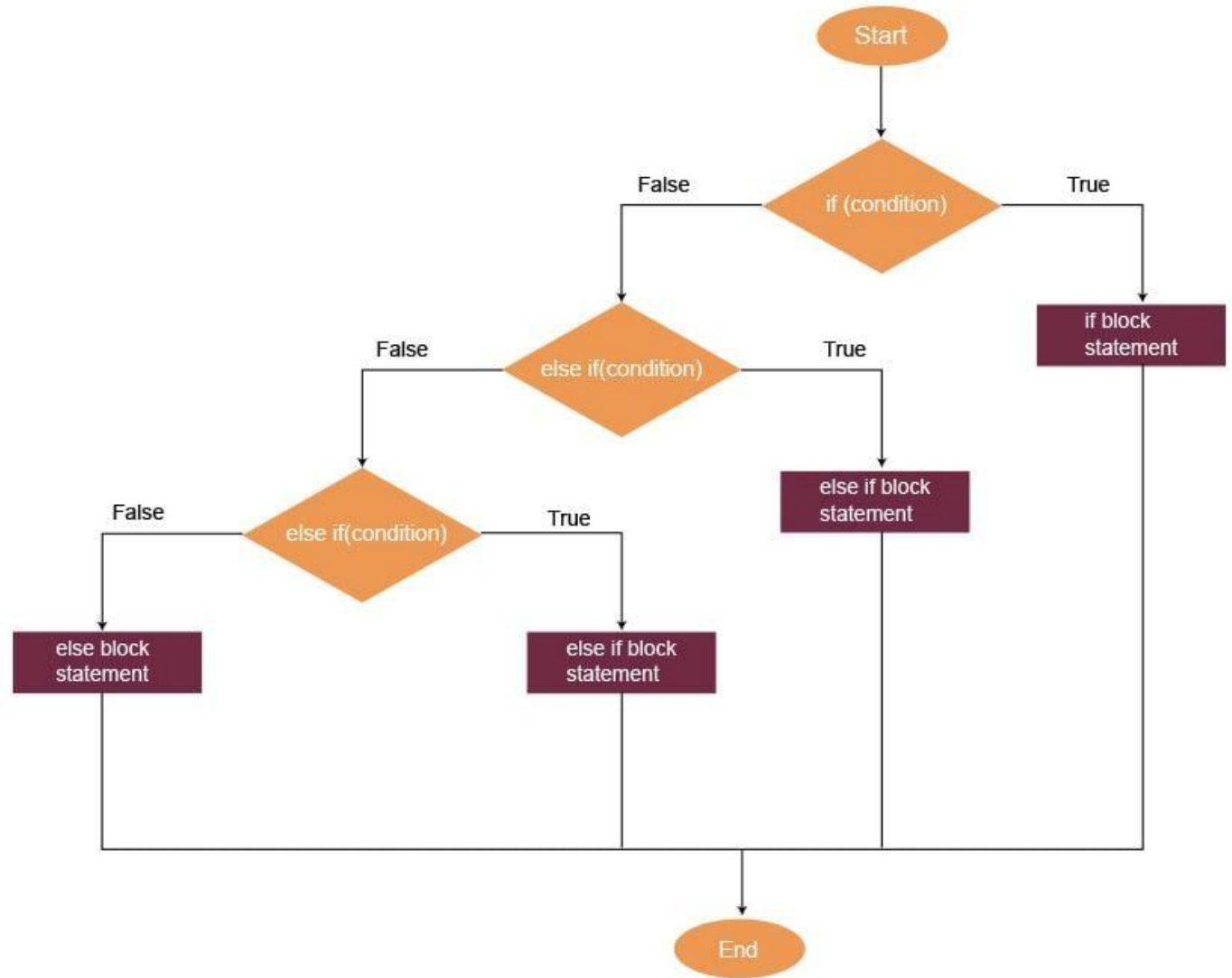
If



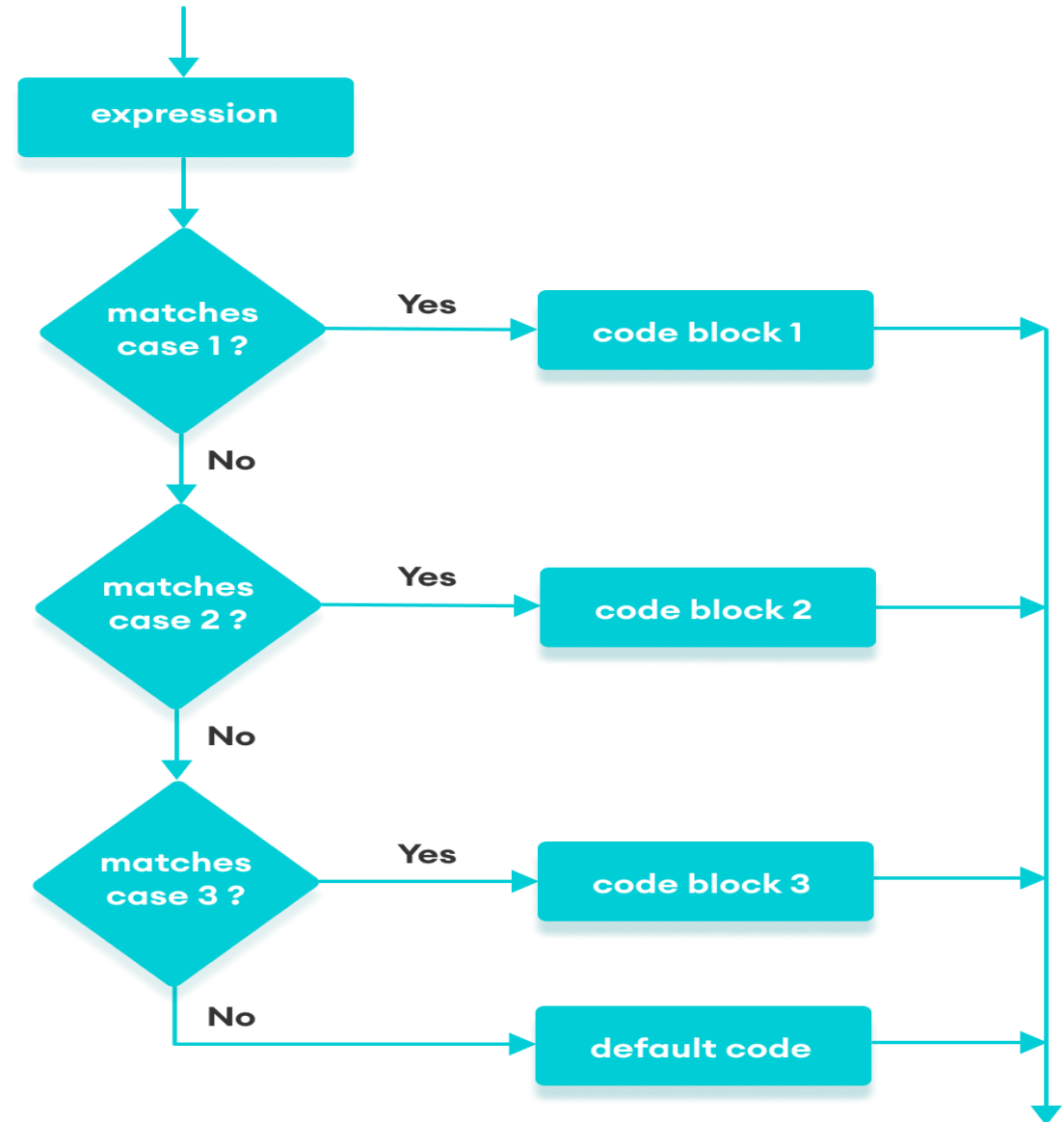
If else



Else-if



switch



Loops

Entry Controlled

for

```
for( initialization ; condition; updation)
{
}

```

while

```
while( condition )
{
}

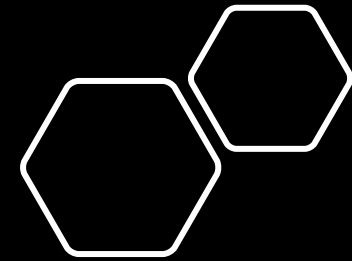
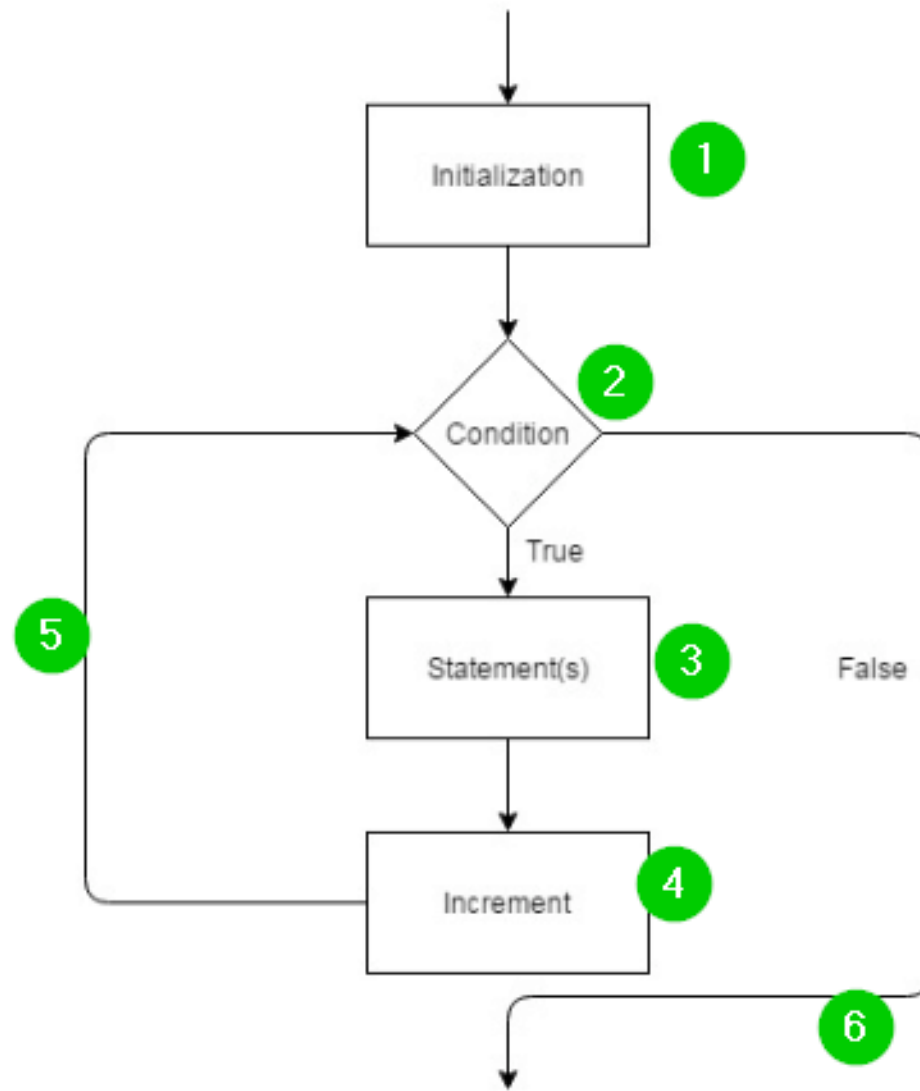
```

Exit Controlled

do-while

```
do
{
}while( condition )

```



3.b) If false

3.a) If true

1.

2.

6.

4.

for (initialization ; condition ; updation)

{

// body of the loop

// statements to be executed

}

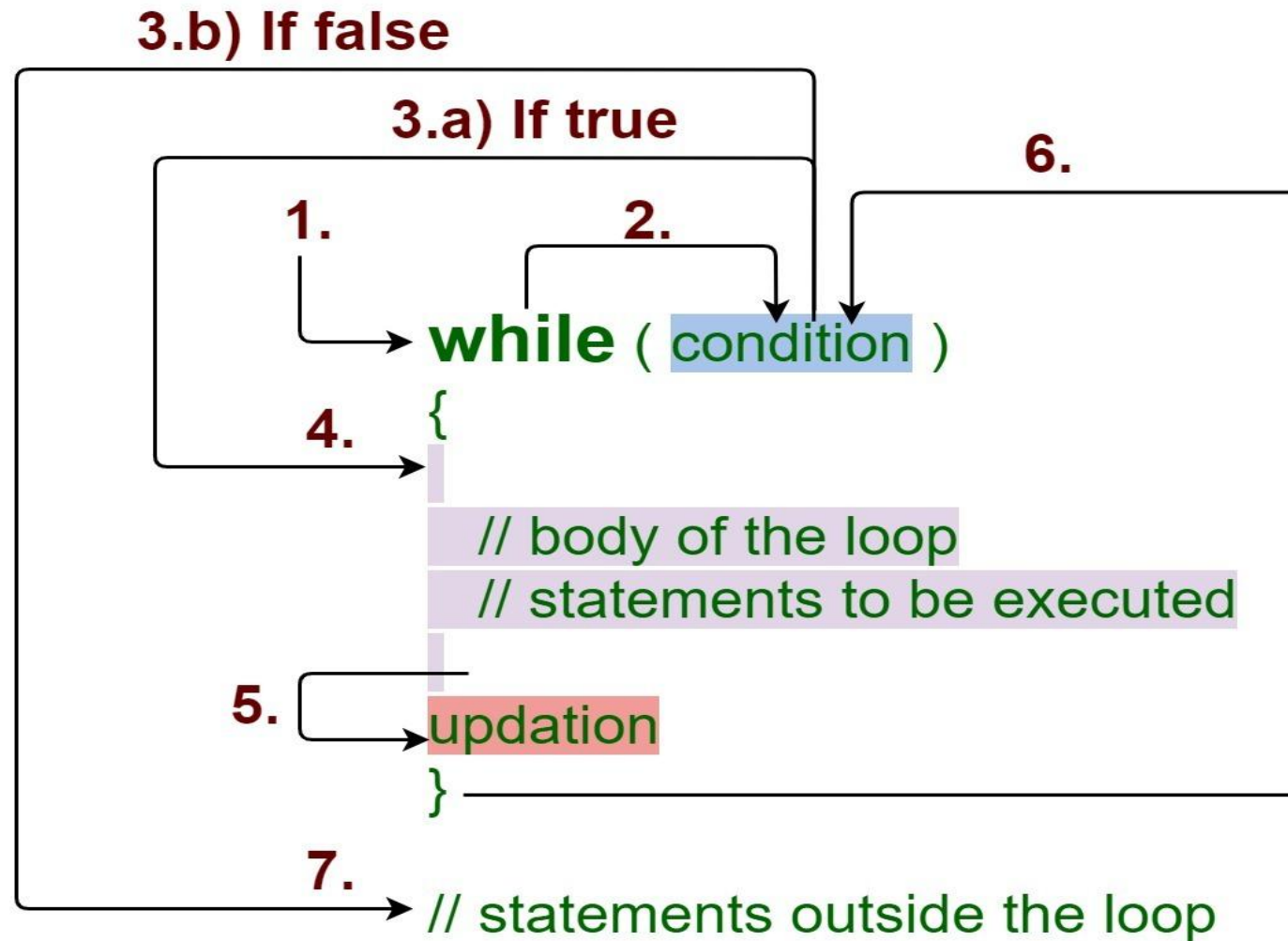
5.

7.

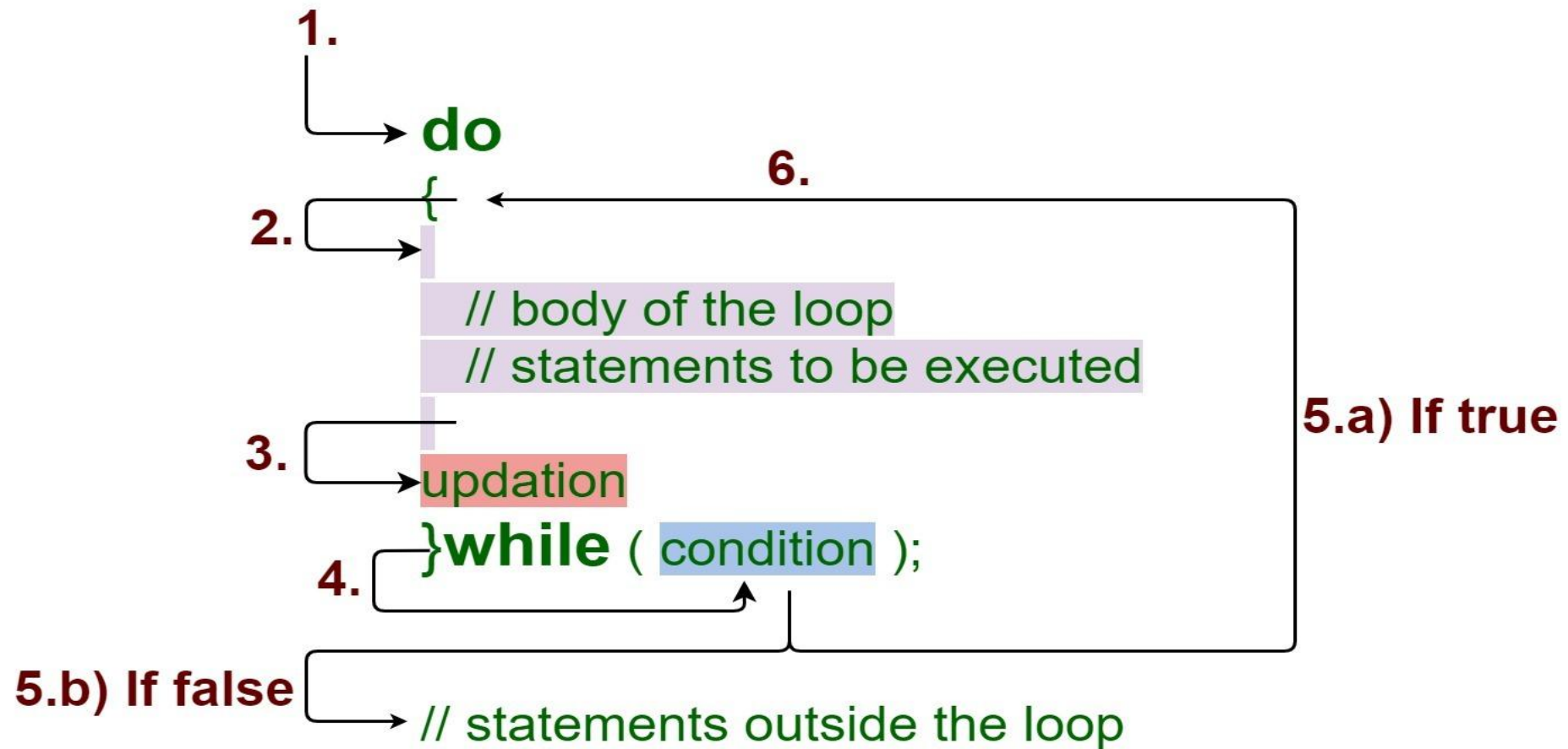
// statements outside the loop

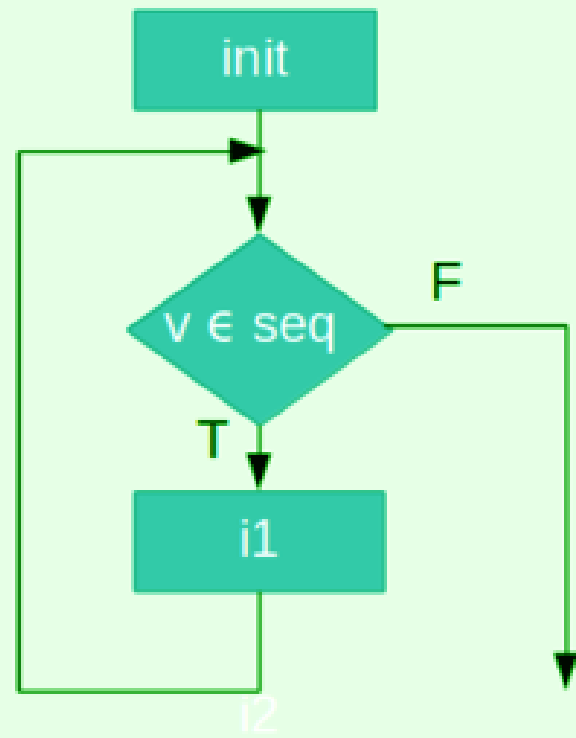


While Loop

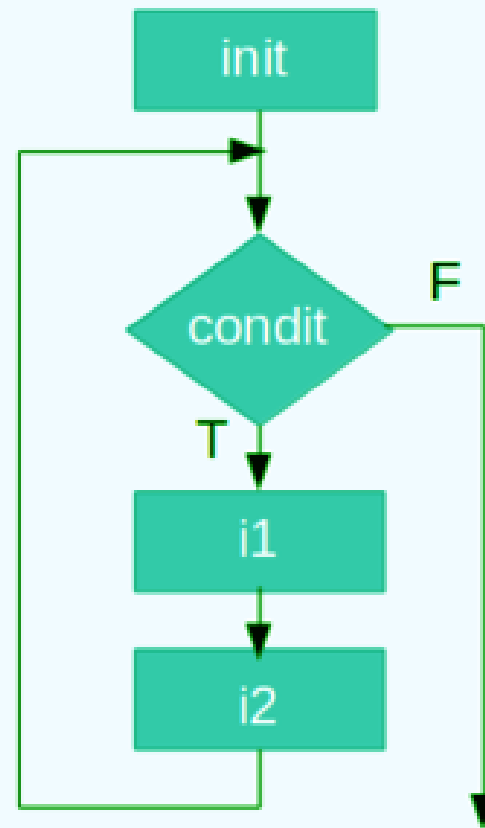


Do - While Loop

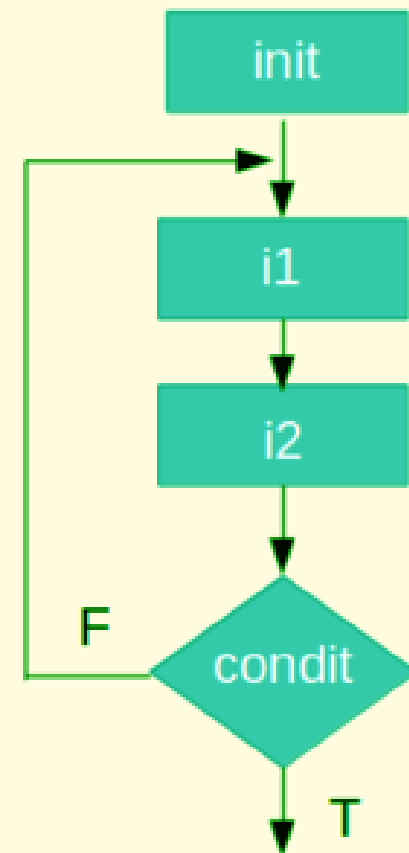




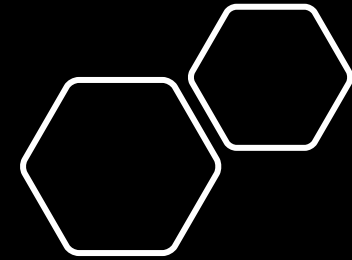
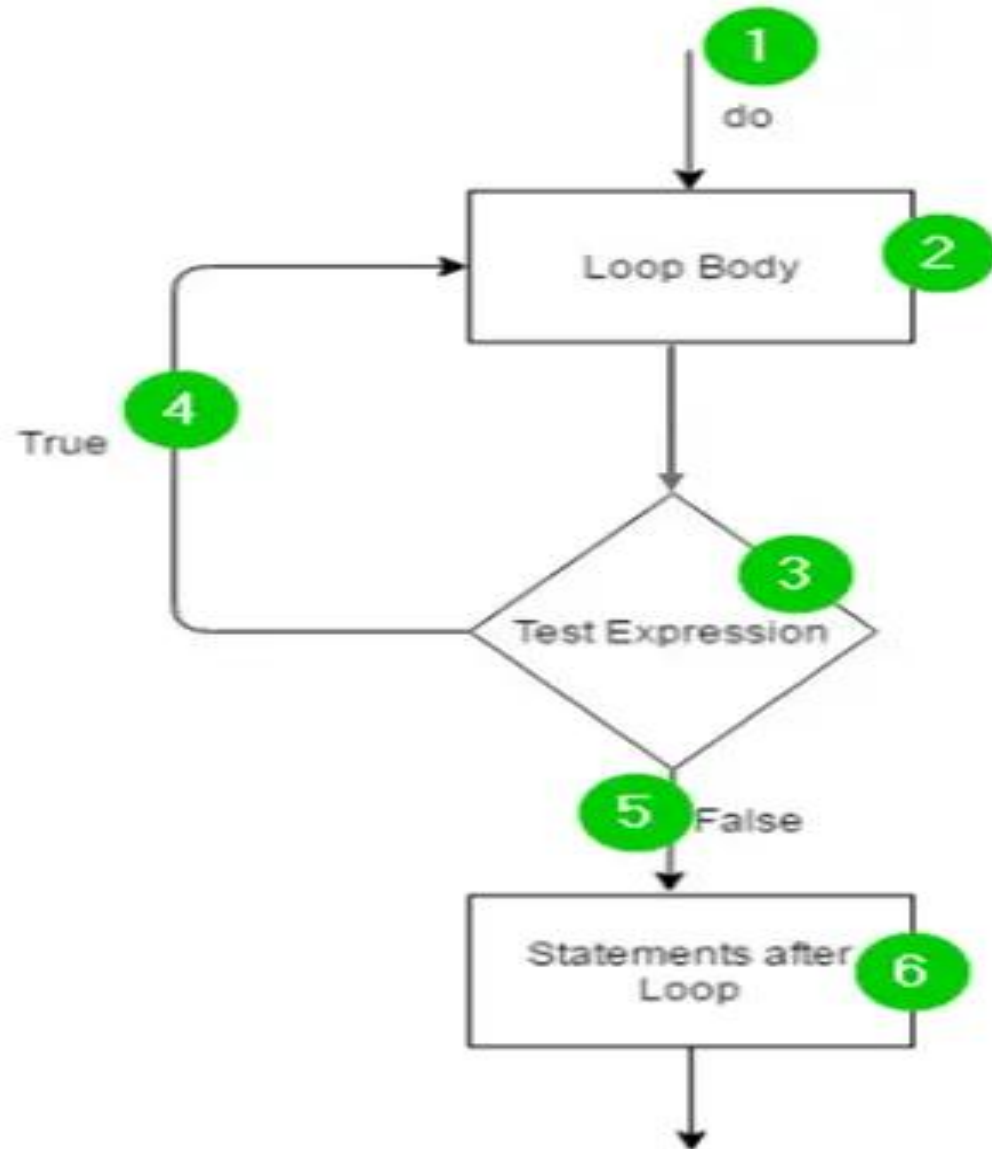
FOR LOOP

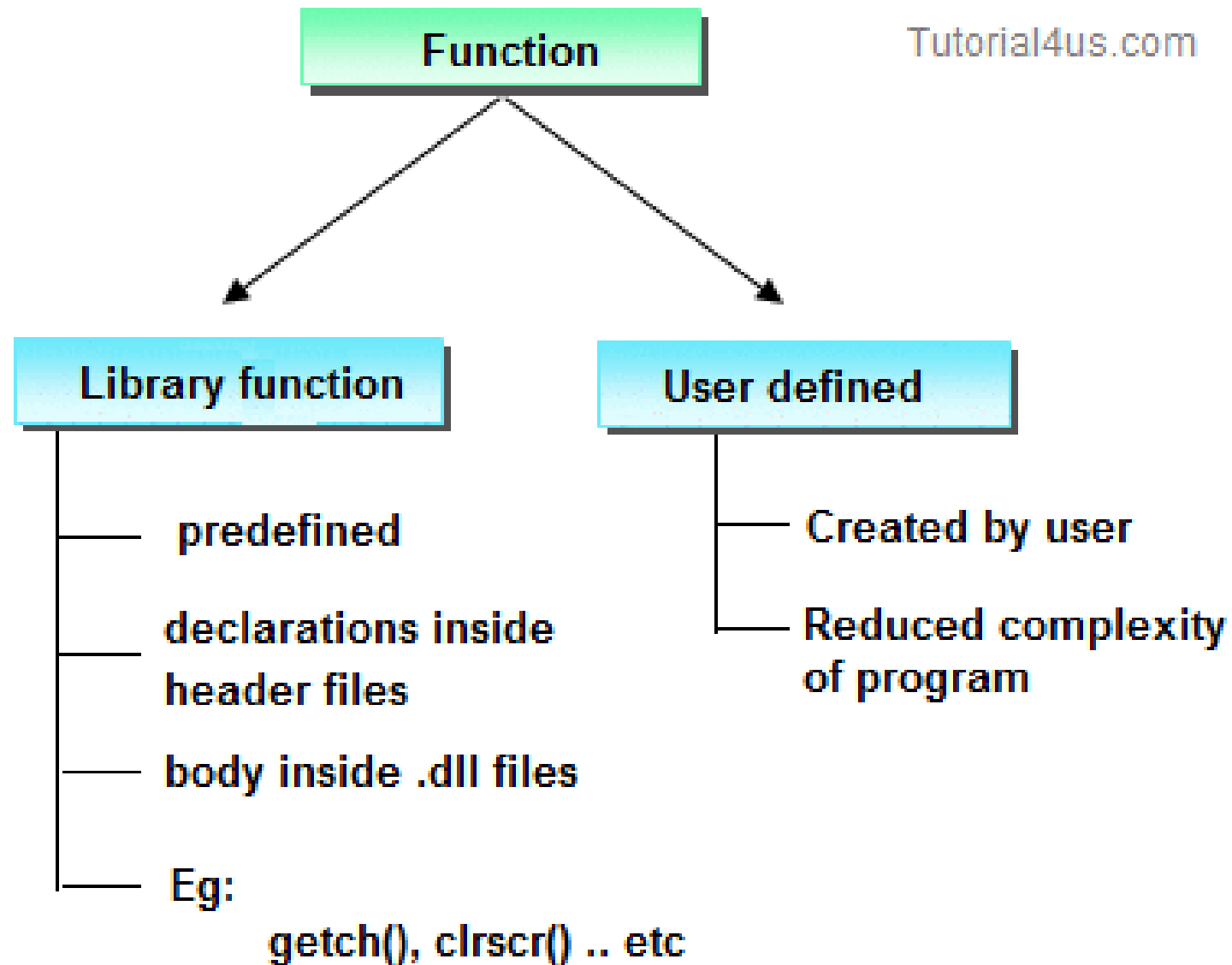


WHILE LOOP



DO WHILE LOOP





Function

The diagram shows a C++ function declaration: `int calculate-sum ( int num1, int num2 ) { int sum = num1 + num2 ; return sum ; }`. Annotations include: 'Return Type' with an arrow pointing to 'int'; 'function name' with an arrow pointing to 'calculate-sum'; 'function parameters' with arrows pointing to '(int num1, int num2)'; and 'function body' with an arrow pointing to the code block between the curly braces.

```
int calculate-sum ( int num1, int num2 )
{
    int sum = num1 + num2 ;
    return sum ;
}
```

```
#include<iostream>
```

```
void displayNum(int n1, double n2) {
```

```
    // code
```

```
}
```

```
int main() {
```

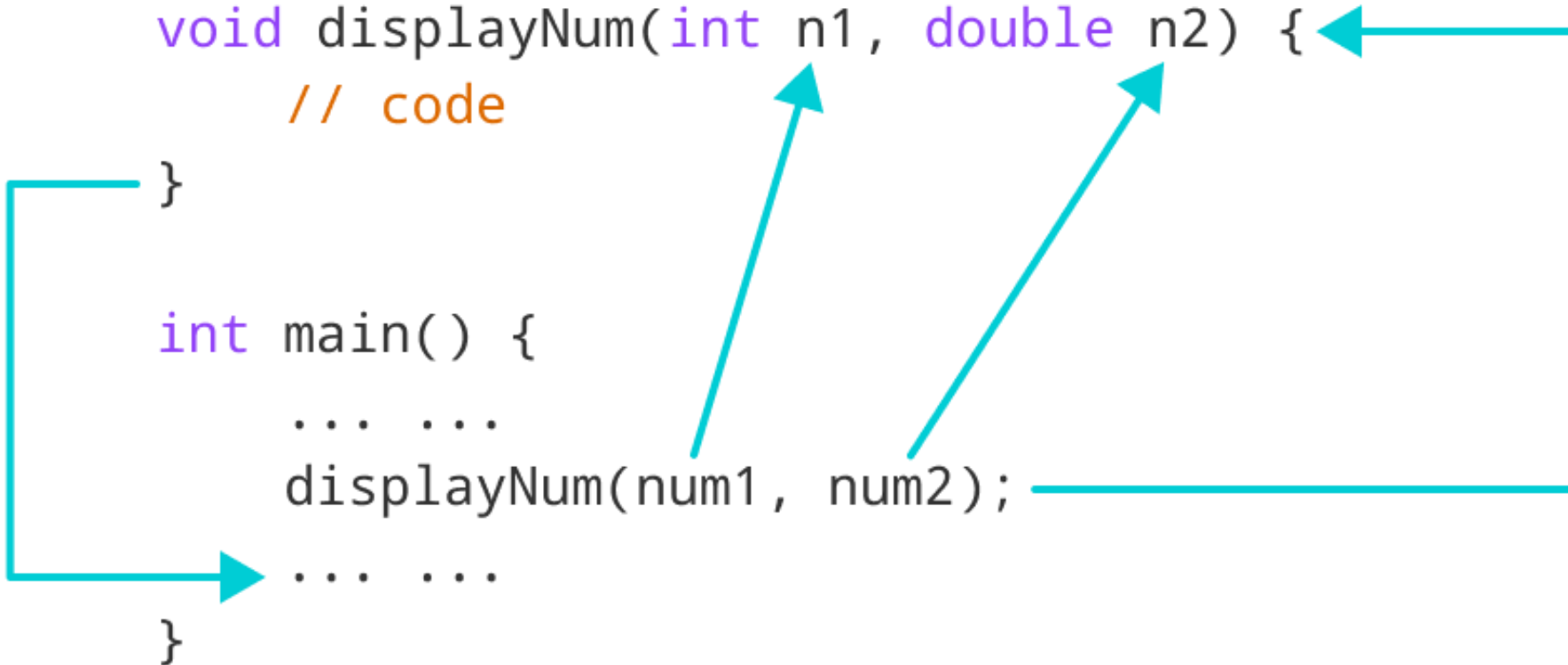
```
    ... ..
```

```
    displayNum(num1, num2);
```

```
    ... ..
```

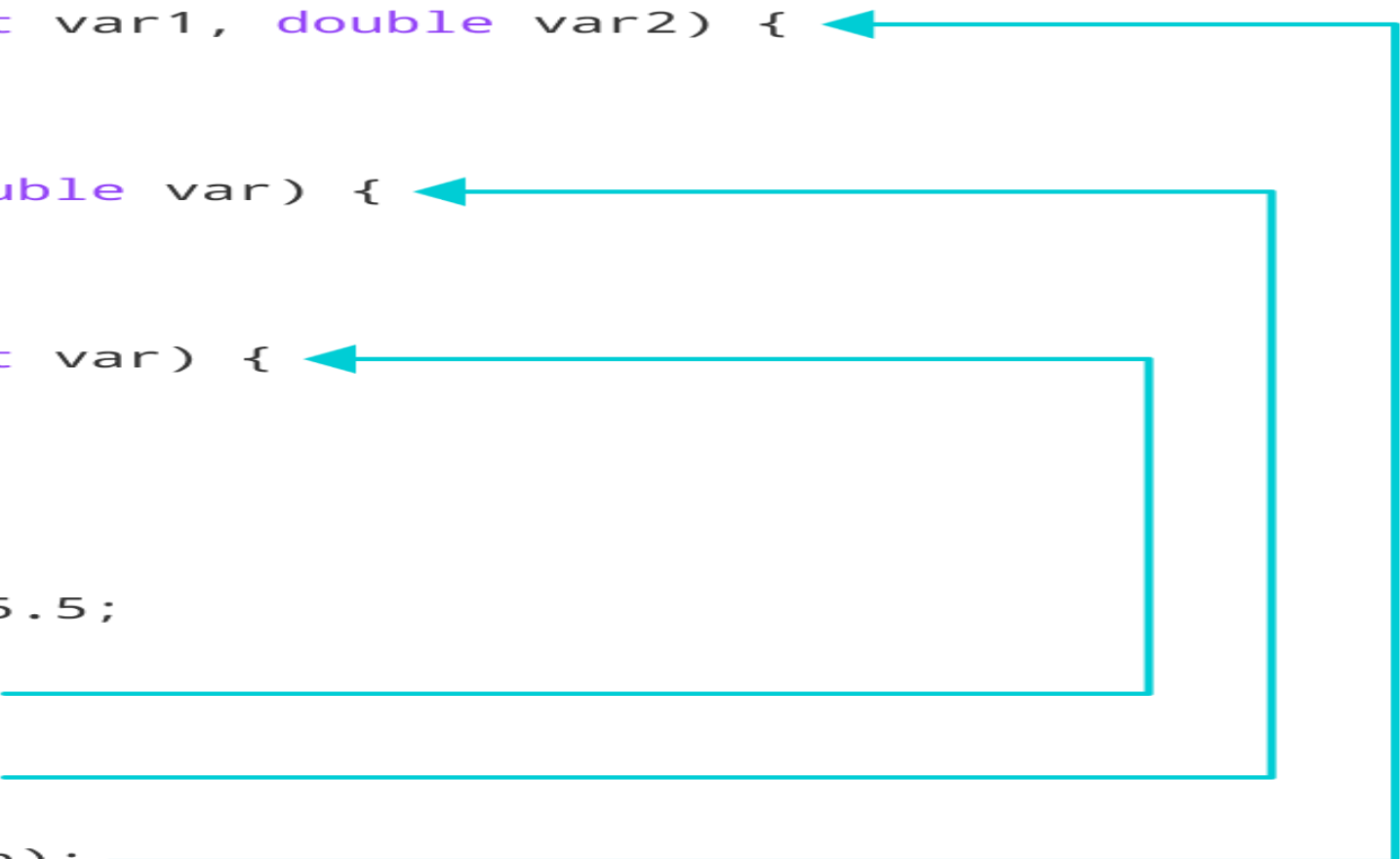
```
}
```

**function
call**



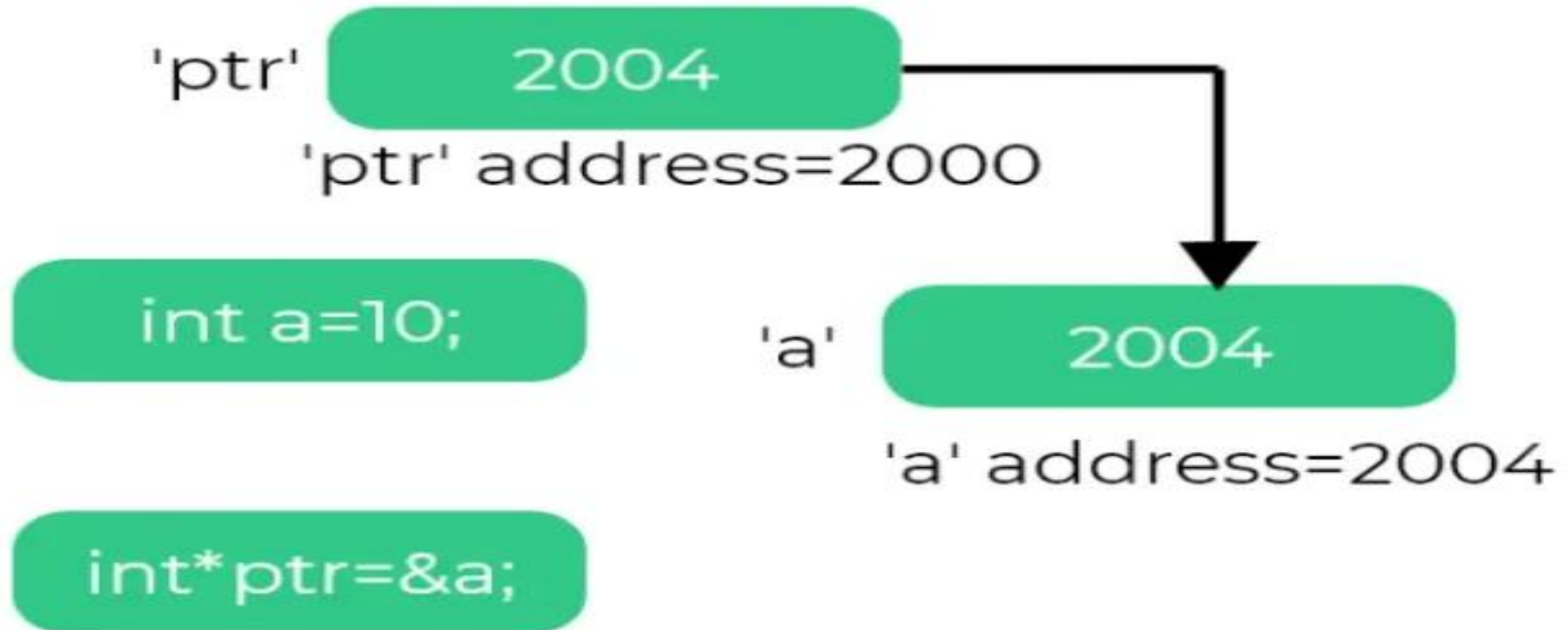
The diagram illustrates a function call. A cyan line originates from the closing curly brace of the `main` function, extends to the right, and then turns upwards to point at the opening curly brace of the `displayNum` function. Two additional cyan arrows originate from the arguments `num1` and `num2` in the `displayNum` call within `main`, pointing to the parameters `n1` and `n2` in the `displayNum` function signature.

```
void display(int var1, double var2) {  
    // code  
}  
  
void display(double var) {  
    // code  
}  
  
void display(int var) {  
    // code  
}  
  
int main() {  
    int a = 5;  
    double b = 5.5;  
  
    display(a);  
    display(b);  
    display(a, b);  
    ...  
}
```



The diagram illustrates the sequence of function calls. Three horizontal cyan lines originate from the right side of the `display(a)`, `display(b)`, and `display(a, b)` calls in the `main` function. These lines extend to the right and then turn left as arrows pointing to the opening curly brace of the corresponding `display` function definitions. The longest line connects `display(a, b)` to the first `display(int var1, double var2)` function. The middle line connects `display(b)` to the `display(double var)` function. The shortest line connects `display(a)` to the `display(int var)` function.

Pointer in C++



Computer		Programmers		
Address	Content	Name	Type	Value
90000000	00	sum	int (4 bytes)	000000FF (255 ₁₀)
90000001	00			
90000002	00			
90000003	FF			
90000004	FF	age	short (2 bytes)	FFFF (-1 ₁₀)
90000005	FF			
90000006	1F			
90000007	FF			
90000008	FF	average	double (8 bytes)	1FFFFFFFFFFFFFFFFF (4.45015E-308 ₁₀)
90000009	FF			
9000000A	FF			
9000000B	FF			
9000000C	FF			
9000000D	FF			
9000000E	90			
9000000F	00			
90000010	00	ptrSum	int* (4 bytes)	90000000
90000011	00			

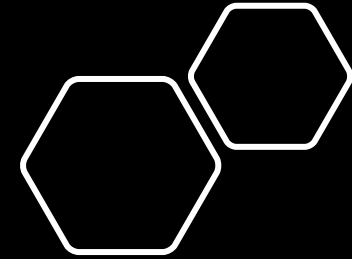
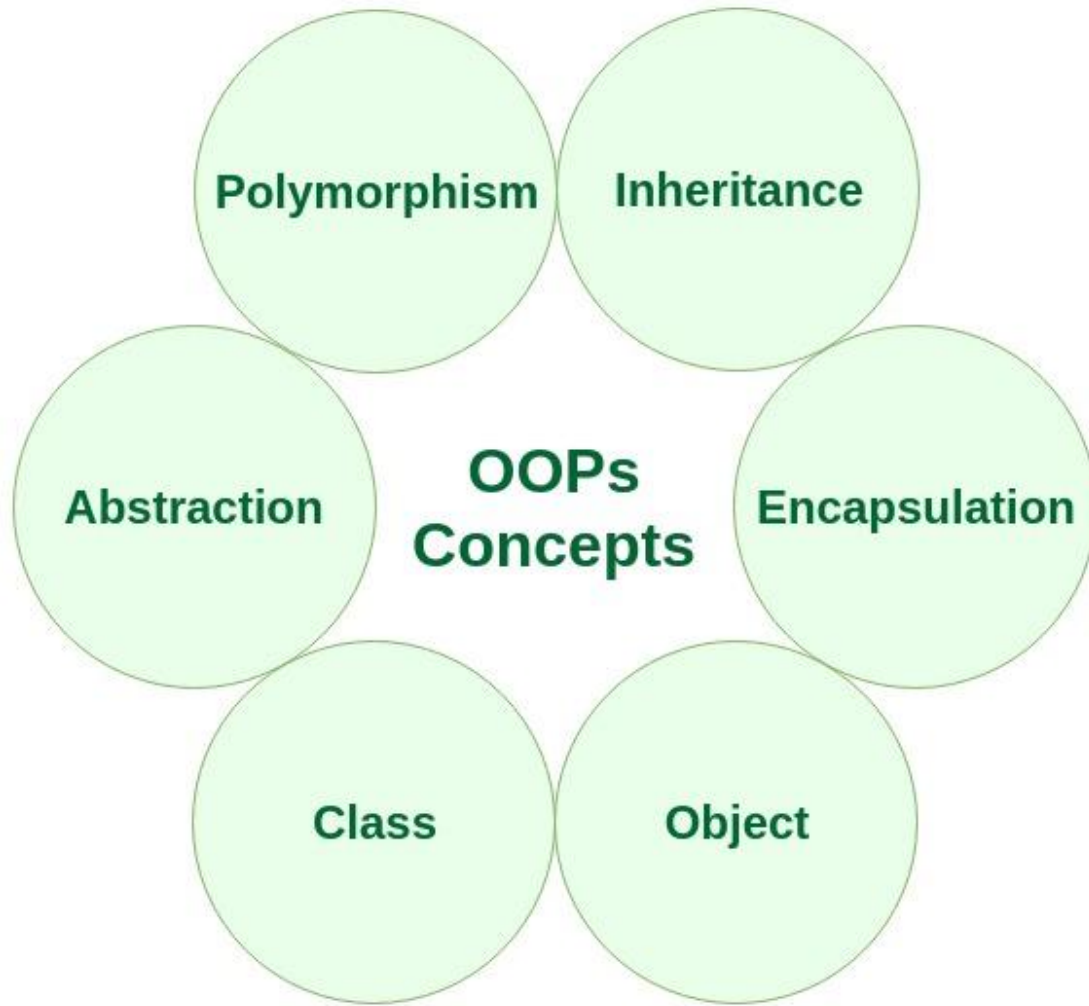
Note: All numbers in hexadecimal

ConsoleApplication2.cpp

ConsoleApplication2

(Global Scope)

```
1      #include <iostream>
2
3      using namespace std;
4
5      int main() {
6          int x = 27;
7          int *ip;
8          ip = &x;
9          cout << "Value of x is : ";
10         cout << x << endl;
11         cout << "Value of ip is : ";
12         cout << ip << endl;
13         cout << "Value of *ip is : ";
14         cout << *ip << endl;
15         return 0;
16     }
```



Benefits of OOPS in C++

Unlock the power of OOPs in C++ by discovering the benefits that it has to offer in the programming language.



Oops Concepts in C++

Master the OOPs concepts in C++ and explore the fundamentals that it offers in C++ programming language with this detailed guide.





Objects

Car

class



Color
Weight
Speed
Model

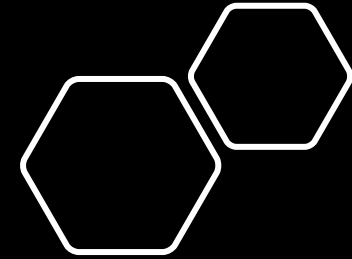
Data members

Common features

startCar()
changeGear()
slowDown()
brake()

method

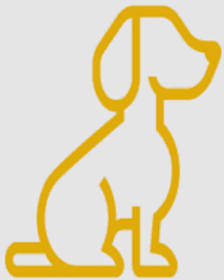
Common functionalities



ANIMAL CLASS



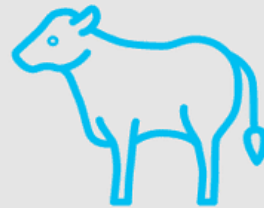
**DOG
CLASS**



**CAT
CLASS**



**COW
CLASS**



To be Continue...

- Thank you for reading these notes. If you find them helpful, **share them with others**, because *sharing is caring* and **knowledge grows when it is shared**.
- If you have any questions, suggestions, or need further assistance, feel free to contact me